AI Contract & Policy Navigator for Small Businesses

1. Concept Overview

The AI Contract & Policy Navigator is a web application designed for small and medium businesses (SMBs), freelancers, and agencies. Users upload vendor contracts, leases, insurance policies, SLAs, and other agreements. The system uses AI to: - Read the document - Identify and highlight risky or important clauses - Explain complex legal language in plain English - Answer specific questions with clause references - Compare multiple versions of a contract side by side - Track renewal and notice dates so users do not miss critical deadlines

The goal is **not** to replace lawyers but to give business owners a powerful pre-review and understanding tool before they negotiate or sign.

2. Why This Opportunity Exists Now

1. Legal AI maturity - Generative AI is already widely adopted for document review and drafting in corporate legal teams. - Vendors and reports note that document review is one of the fastest-growing generative AI use cases in legal and compliance.

2. SMB gap - Enterprise Contract Lifecycle Management (CLM) systems focus on big organizations with in-house counsel and complex workflows. - Small businesses rarely have budgets or time for CLM; most contracts are never reviewed by a lawyer. - A lightweight, self-serve, AI-powered reviewer aimed at non-lawyers is still under-served.

3. Market & risk - A single bad clause (auto-renew, liability, penalty) can cost an SMB more than a year of subscription fees. - This creates a clear willingness to pay for a tool that catches issues early and explains them simply.

3. Target Users and Use Cases

Primary user groups:

- Agencies and freelancers • Marketing, design, development agencies signing MSAs, NDAs, SOWs. • Freelancers signing one-off client contracts and platform agreements.

- Product and SaaS companies • Signing vendor contracts for cloud, analytics, tools, office leases. • Negotiating SLAs with customers.

- Traditional SMBs • Retail, logistics, manufacturing, distributors signing supply, franchise, and lease agreements. • Owners dealing with insurance policies and service contracts.

"Jobs to be done":

1. Understand a contract before signing it. 2. Quickly check if a renewal or new version is worse than the old one. 3. Know key dates (renewal, termination notice, penalties). 4. Prepare talking points for negotiation with the vendor or lawyer.

4. Core Features in Detail

4.1 Risk Radar View

User uploads a contract PDF/DOC and, after processing, sees a dashboard with:

- Overall risk summary: a high-level score (low/medium/high) with a brief explanation. - Categories: • Term & renewal (initial term, auto-renew, notice period) • Termination (termination for convenience, for cause, early termination fees) • Payment (fees, late payment penalties, price increase mechanisms) • Liability & indemnity (liability caps, exclusions, indemnification) • Data & privacy (data processing, security, breach notification) • IP & confidentiality (ownership, license grants, non-compete, NDA)

Each category lists detected clauses with: - Clause title / type (e.g., "Auto-Renew", "Limitation of Liability"). - A short plain-language explanation. - A severity tag with a simple rationale (e.g., "High – unlimited liability with no cap").

4.2 Q&A with Clause References

The Q&A module lets the user ask natural language questions like: - "What happens if I terminate before the end of the term?" - "Am I responsible if their servers are hacked?" - "Can they increase prices whenever they want?"

The system: - Performs semantic search over the contract to locate relevant clauses. - Feeds those clauses into a language model. - Returns a concise, plain-language answer with explicit references, such as: "If you terminate before the Initial Term ends, you must pay all remaining monthly fees (see Clause 5.2)."

This combination of semantic search + retrieval-augmented generation (RAG) is essential to avoid hallucinations.

4.3 Multi-Document Comparison ("What Changed?")

Use case: a vendor sends a new version of a contract; the user wants to quickly see what changed.

Workflow: - User uploads the previous contract (v1) and the new contract (v2). - The system aligns clauses using similarity and headings. - It highlights differences in key areas: • Liability cap changed (e.g., 12 months of fees to 24 months, or to unlimited). • New auto-renew clause added. • Notice period for termination changed. • New data-sharing or sub-processor rights introduced.

UI features: - Side-by-side view of v1 vs v2. - Summary text: "Compared to your previous contract, this version introduces an auto-renew clause and reduces your termination notice period from 60 days to 30 days."

4.4 Renewal & Date Tracking

From parsed metadata, the system extracts: - Effective date. - Initial term and renewal term. - Key notice dates (e.g., "must give non-renewal notice 60 days before end of term").

Then it: - Shows a timeline with upcoming important dates. - Allows the user to set reminders (email/WhatsApp) for non-renewal or review.

4.5 Optional v2 Features

- Market-standard hints: label certain clauses as "stricter than typical" or "favorable" based on internal benchmarks. - Collaboration: share annotated contracts with teammates or external lawyers.

5. High-Level Architecture ----------

At a high level, the system has four layers:

1. Ingestion and Extraction Layer 2. Analysis and Structuring Layer 3. AI Reasoning Layer (RAG + Q&A) 4. Web Application Layer (Frontend + Backend APIs)

5.1 Ingestion and Extraction

- File upload API • Accepts PDF, DOCX, possibly image scans. - Storage • Save raw files to object storage (e.g., S3-compatible bucket). - Text extraction • Use a PDF/DOCX parser; optionally run OCR for scanned documents. - Chunking • Split into logical sections/clauses using headings and numbering when possible.

5.2 Analysis and Structuring

- Clause classification • Use keyword heuristics + AI classification to label clauses (Termination, Liability, etc.). - Field extraction • For each clause, extract key parameters such as dates, notice periods, limits, amounts, and parties. - Data storage • Store structured data in a relational database (Postgres/Prisma). • Store text chunks and embeddings in a vector database for semantic search.

5.3 AI Reasoning Layer

- Semantic search • Given a user question, retrieve the most relevant clauses via vector similarity. - RAG pipeline • Provide retrieved clauses + question to a language model with a prompt that enforces: - Answer only from provided text. - Cite clause numbers and headings. - Use simple language for non-lawyers. - Risk scoring • Apply rule-based logic and/or AI-based scoring to compute severity of each clause.

5.4 Web Application Layer

Backend (e.g., FastAPI): - Auth, sessions, user management. - Endpoints for: • Upload and processing. • Fetching risk summaries and clause details. • Q&A requests. • Comparison and timeline data.

Frontend (e.g., Next.js/React): - Upload page and processing status. - Risk radar dashboard. - Q&A chat-like interface. - Comparison view and timelines.

6. MVP Scope (First 4–6 Weeks) ----------

The key to execution is to start with a very narrow, realistic MVP and expand later.

6.1 MVP Constraints

- Focus on 1–2 contract types first: • B2B SaaS subscription agreements. • Simple

NDAs. - English-only contracts. - One user persona: "small agency owner" or "small SaaS founder".

6.2 Week-by-Week Plan (Example)

Week 1: Foundations - Define concrete MVP scope: which clauses and which risks you want to detect. - Set up project structure: backend (FastAPI), frontend, database. - Implement basic file upload and storage.

Week 2: Extraction & Storage - Implement text extraction and chunking from PDF/DOCX. - Design relational schema for contracts, clauses, and extracted fields. - Seed the system with a few sample contracts to develop against.

Week 3: Clause Classification & Risk Radar v1 - Implement simple heuristics and LLM calls to classify clauses into types. - Extract basic parameters (e.g., term length, auto-renew presence, liability cap amount). - Build the first version of the risk radar UI.

Week 4: Semantic Q&A - Add vector search for clauses. - Build the Q&A API using a RAG pattern. - Implement the Q&A UI and show clause references in answers.

Week 5: Comparison View - Implement upload of v1 + v2 contracts. - Align clauses using similarity and generate a "what changed" summary. - Create a side-by-side comparison UI.

Week 6: Polish and First User Tests - Improve prompts and clause grouping. - Add simple reminders for renewal dates. - Run the tool with real contracts from friendly users; collect feedback.

7. Execution Checklist ----------

To keep yourself on track, treat this as a project with clear milestones:

- [ ] Choose primary user persona and contract type. - [ ] Set up repo, CI, basic deployment environment. - [ ] Implement upload + extraction + DB storage. - [ ] Implement clause classification and risk radar v1. - [ ] Implement semantic Q&A. - [ ] Implement basic comparison functionality. - [ ] Add date extraction and simple renewal reminders. - [ ] Test with real users and refine based on feedback.

8. Risk, Limitations, and Ethics ----------

- Legal disclaimer • The tool is an assistant, not a lawyer. Always include clear disclaimers. - Accuracy • AI can misinterpret clauses; design the UX to show raw clauses and encourage users to double-check. - Data privacy • Contracts often contain sensitive information. Use encryption, secure storage, and clear privacy policies.

If you execute the above plan, you will have a focused, differentiated AI product that demonstrates real-world generative AI integration, retrieval, UX, and business understanding.